

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Инфокоммуникационных технологий**

Образовательная программа **Мобильные и облачные технологии**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

Лабораторная работа №1

По дисциплине «Методы оптимизации»

Тема: Решение задач линейного программирования

Выполнил	Езиева М.Д. К3339 1.2
Проверил	Лаврентьев С.М.
Дата	<u>05.10.2026</u>

Санкт-Петербург 2026

Вариант 8

$$Z = x_1 + 3x_2 + 2x_3 + x_4 \rightarrow \min$$

$$\begin{cases} x_1 + x_2 + 2x_4 \leq 8 \\ x_2 + x_3 + x_4 = 6 \\ 2x_1 + x_3 \geq 2 \end{cases}$$

$$x_1, x_2, x_3, x_4 \geq 0$$

1. Решение вручную с помощью симплекс-метода

Решение:
Вариант 8

$$Z = x_1 + 3x_2 + 2x_3 + x_4 \rightarrow \min$$

$$\begin{cases} x_1 + x_2 + 2x_4 \leq 8 \\ x_2 + x_3 + x_4 = 6 \\ 2x_1 + x_3 \geq 2 \\ x_1, \dots, x_4 \geq 0 \end{cases}$$

Приводим к каноническому виду

$$\begin{cases} x_1 + x_2 + 2x_4 + x_5 = 8 \\ x_2 + x_3 + x_4 = 6 \\ 2x_1 + x_3 - x_6 = 2 \\ x_1, \dots, x_6 \geq 0 \end{cases}$$

Вспомогательная задача

$$W' = x_7 + x_8 \rightarrow \min$$

$$\begin{cases} x_1 + x_2 + 2x_4 + x_5 = 8 \quad (1) \\ x_2 + x_3 + x_4 + x_7 = 6 \quad (2) \\ 2x_1 + x_3 - x_6 + x_8 = 2 \quad (3) \\ x_1, \dots, x_8 \geq 0 \end{cases} \quad x_5, x_7, x_8 - \text{базисные переменные}$$

Из (2) выразим: $x_7 = 6 - x_2 - x_3 - x_4$

Из (3) выразим: $x_8 = 2 - 2x_1 - x_3 + x_6$

Подставим в W' : $W' = 6 - x_2 - x_3 - x_4 + 2 - 2x_1 - x_3 + x_6$

$$W' = -2x_1 - x_2 - 2x_3 - x_4 + x_6 + 8 \rightarrow \min$$

Свободные переменные: x_1, x_2, x_3, x_4, x_6

Базисные переменные: x_5, x_7, x_8

Базис	x_1	x_2	x_3	x_4	x_6	b
x_5	1	1	0	2	0	8
x_7	0	1	1	1	0	6
x_8	2	0	1	0	-1	2
P	-2	-1	-2	-1	1	-8

Пересчитываем разрешающий элемент и столбец

Базис	x_8	x_2	x_3	x_4	x_6	b
x_5	$-\frac{1}{2}$	1	0	2	0	8
x_7	0	1	1	1	0	6
x_1	$\frac{1}{2}$	0	$\frac{1}{2}$	0	$-\frac{1}{2}$	1
P	1	-1	-2	-1	1	-8

Базис	x_8	x_2	x_3	x_4	x_6	b
x_5	$-\frac{1}{2}$	1	$-\frac{1}{2}$	2	$\frac{1}{2}$	7
x_7	0	1	1	1	0	6
x_1	$\frac{1}{2}$	0	$\frac{1}{2}$	0	$-\frac{1}{2}$	1
P	1	-1	-1	-1	0	-6

Замена базиса $x_7 \leftrightarrow x_2$

Базис	x_8	x_7	x_3	x_4	x_6	b
x_5	$-\frac{1}{2}$	-1	$-\frac{3}{2}$	1	$\frac{1}{2}$	1
x_2	0	1	1	1	0	6
x_1	$\frac{1}{2}$	0	$\frac{1}{2}$	0	$-\frac{1}{2}$	1
P	1	1	0	0	0	0

В строке P отсутствуют отрицательные элементы $\Rightarrow$ достигнуто оптимальное решение
 Вспомогательная задача: $W'_{\min} = 0 \Rightarrow$ ДРЗ исходной задачи не пустая

Переходим к решению основной задачи

Разрешающий элемент: 2

Замена базиса $x_8 \leftrightarrow x_1$

Новый разреш. элемент равен обратному старому:

$$\hat{a}_{i^*j^*} = \frac{1}{a_{i^*j^*}}$$

Новый элемент: $\frac{1}{2}$

Пересчитываем разреш. строки x_8

$$x_2: \frac{0}{2} = 0; x_3: \frac{1}{2}; x_4: \frac{0}{2} = 0; x_6: \frac{-1}{2}; b: \frac{2}{2} = 1$$

Пересчитываем разреш. столбец:

$$\hat{a}_{ij^*} = -\frac{a_{ij^*}}{a_{i^*j^*}}$$

$$x_5: -\frac{1}{2}; x_7: \frac{-0}{2} = 0; P: \frac{-2}{2} = -1$$

Пересчитываем остальные элементы

$$\hat{a}_{ij} = a_{ij} - \frac{a_{i^*j} \cdot a_{ij^*}}{a_{i^*j^*}}$$

Например

$$a_{5,2} = 1 - \frac{0 \cdot 1}{2} = 1$$

$$\hat{b}_i = b_i - \frac{b_{i^*} \cdot a_{ij^*}}{a_{i^*j^*}}$$

Например b для строки x_5

$$b = 8 - \frac{2 \cdot 1}{2} = 7$$

Выбираем искусственные переменные

Базис	x_3	x_4	x_6	b
x_5	$-\frac{3}{2}$	1	$\frac{1}{2}$	1
x_2	1	1	0	6
x_1	$\frac{1}{2}$	0	$-\frac{1}{2}$	1

Базис x_5, x_2, x_1

Свободные x_3, x_4, x_6

Исходная функция:

$$Z = x_1 + 3x_2 + 2x_3 + x_4 \rightarrow \min$$

Нужно выразить целевую функцию только через свободные переменные

Из таблицы:

$$\frac{1}{2}x_3 - \frac{1}{2}x_6 + x_1 = 1$$

$$x_1 = 1 - \frac{1}{2}x_3 + \frac{1}{2}x_6$$

$$x_3 + x_4 + x_2 = 6$$

$$x_2 = 6 - x_3 - x_4$$

Подставляем:

$$Z = 1 - \frac{1}{2}x_3 + \frac{1}{2}x_6 + 3(6 - x_3 - x_4) + 2x_3 + x_4$$

$$Z = 1 - \frac{1}{2}x_3 + \frac{1}{2}x_6 + 18 - 3x_3 - 3x_4 + 2x_3 + x_4$$

$$Z = -\frac{3}{2}x_3 - 2x_4 + \frac{1}{2}x_6 + 19 \rightarrow \min$$

Таблица основной задачи:

(1)

Базис	x_3	x_4	x_6	b
x_5	$-\frac{3}{2}$	1	$\frac{1}{2}$	1
x_2	1	1	0	6
x_1	$\frac{1}{2}$	0	$-\frac{1}{2}$	1
P	$-\frac{3}{2}$	-2	$\frac{1}{2}$	-19

Замена

базиса $x_5 \leftrightarrow x_4$

→

(2)

Базис	x_3	x_5	x_6	b
x_4	$-\frac{3}{2}$	1	$\frac{1}{2}$	1
x_2	$\frac{5}{2}$	-1	$-\frac{1}{2}$	5
x_1	$\frac{1}{2}$	0	$-\frac{1}{2}$	1
P	$-\frac{9}{2}$	2	$\frac{3}{2}$	-17

Замена базиса $x_2 \leftrightarrow x_3$

(3)

Базис	x_2	x_5	x_6	b
x_4	$\frac{2}{5}$	$\frac{2}{5}$	$\frac{1}{5}$	4
x_3	$\frac{2}{5}$	$-\frac{2}{5}$	$-\frac{1}{5}$	2
x_1	$-\frac{1}{5}$	$\frac{1}{5}$	$-\frac{2}{5}$	0
P	$\frac{9}{5}$	$\frac{1}{5}$	$\frac{3}{5}$	-8

в строке P отсутствуют отрицательные элементы $\Rightarrow$ найденное решение является оптимальным

Свободные элементы $x_2 = x_5 = x_6 = 0$

Базисные $x_4 = 4$; $x_3 = 2$; $x_1 = 0$

Значит для исходных четырех переменных

$$x_1 = 0; x_2 = 0; x_3 = 2; x_4 = 4$$

$$-Q = -8 \Rightarrow Z_{\min} = 8$$

Ответ: $X^* = (0; 0; 2; 4)$, $Z_{\min} = 8$

2. Решение в Excel

Сначала в таблицу были внесены переменные x_1, x_2, x_3, x_4 коэффициенты целевой функции и коэффициенты ограничений.

Также для каждого ограничения были рассчитаны значения его левой части. В качестве начальных значений переменных были установлены нули.

Переменные	X1	X2	X3	X4		
	0	0	0	0		
Целевая функция	0					
Ограничения	1	1	0	2	8	
	0	1	1	1	6	
	2	0	1	0	2	

Настройка параметров надстройки «Поиск решения»

Параметры поиска решения

Оптимизировать целевую функцию:

До: ☐ Макс ☒ Мин ☐ Значения:

Изменяя ячейки переменных:

В соответствии с ограничениями:

\$F\$6 <= 8

\$F\$7 = 6

\$F\$8 >= 2

Добавить

Изменить

Удалить

Сбросить

Загрузить/сохранить

☒ Сделай переменные без ограничений неотрицательными

Выберите метод решения:

Метод решения

Для гладких нелинейных задач используйте поиск решения нелинейных задач методом ОПГ, для линейных задач – поиск решения линейных задач симплекс-методом, а для негладких задач – эволюционный поиск решения.

Итог

Оптимальная точка имеет вид:

$$x^* = (0; 0; 2; 4),$$

а минимальное значение целевой функции составляет:

$$Z_{\min} = 8.$$

Полученный в MS Excel результат совпадает с результатом решения задачи симплекс-методом вручную

Переменные	X1	X2	X3	X4		
		0	0	2	4	
Целевая функция		8				
Ограничения		1	1	0	2	8
		0	1	1	1	6
		2	0	1	0	2

3. Реализация кодом

Для решения задачи была написана программа на языке Python, реализующая симплекс-метод. Для работы с матрицами и выполнения вычислений использовалась библиотека NumPy.

Программа написана в общем виде, исходные данные задачи задаются отдельно: коэффициенты целевой функции, матрица ограничений, знаки ограничений, правые части и направление оптимизации (min или max). В качестве исходных данных передаются коэффициенты целевой функции c, матрица коэффициентов ограничений A, знаки ограничений signs, правые части b и направление оптимизации sense.

Сначала программа приводит ограничения к каноническому виду. Для приведения ограничений к каноническому виду была написана функция `canonical_form()`. Она проверяет знак правой части ограничения,

добавляет дополнительные переменные и при необходимости формирует искусственный базис

```
def canonical_form(A, b, signs):
    """
    Приведение системы ограничений к каноническому виду
    """

    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    signs = list(signs)

    # Если b < 0, умножаем всё ограничение на -1
    for i in range(len(b)):
        if b[i] < 0:
            A[i] *= -1
            b[i] *= -1

            if signs[i] == "<=":
                signs[i] = ">="
            elif signs[i] == ">=":
                signs[i] = "<="

    m, n = A.shape

    names = [f"x{i + 1}" for i in range(n)]
    basis = [-1] * m
    artificial = []

    # Добавляем дополнительные переменные
    for i, sign in enumerate(signs):

        if sign in ("<=", ">="):

            column = np.zeros((m, 1))

            if sign == "<=":
                column[i] = 1
            else:
                column[i] = -1

            A = np.hstack((A, column))

            names.append(f"x{len(names) + 1}")

            # При <= новая переменная сразу базисная
            if sign == "<=":
                basis[i] = A.shape[1] - 1
```

```

# Добавляем искусственные переменные, если в строке ещё нет базисной переменной
for i in range(m):

    if basis[i] == -1:

        column = np.zeros((m, 1))
        column[i] = 1

        A = np.hstack((A, column))

        names.append(f"x{len(names) + 1}")

        index = A.shape[1] - 1

        basis[i] = index
        artificial.append(index)

return A, b, basis, artificial, names

```

В результате функция приводит ограничения к равенствам и формирует начальный базис. Для ограничений, где обычную базисную переменную получить нельзя, автоматически добавляются искусственные переменные

На первом шаге минимизируется сумма искусственных переменных. Если в результате получается $W_{\min} = 0$ искусственные переменные можно удалить и перейти к исходной целевой функции. Если $W_{\min} > 0$, исходная система ограничений несовместна и допустимого решения не существует.

Сам процесс симплекс-итераций реализован в функции `simplex()`:

```

def simplex(T, basis):

    while True:

        # Ищем входящую переменную: первый отрицательный коэффициент
        entering = None

        for j, value in enumerate(T[-1, :-1]):
            if value < -EPS:
                entering = j
                break

```

```

# Отрицательных коэффициентов нет значит найден оптимум
if entering is None:
    return T, basis

# Выбираем выходящую переменную
ratios = []

for i in range(len(basis)):

    if T[i, entering] > EPS:

        ratio = T[i, -1] / T[i, entering]

        ratios.append((ratio, i))

# Если положительных элементов в столбце нет, целевая функция не ограничена
if not ratios:
    raise Unbounded(
        "Целевая функция не ограничена."
    )

_, leaving = min(ratios)

# Разрешающий элемент
pivot = T[leaving, entering]

# Делим разрешающую строку
T[leaving] /= pivot

# Обнуляем остальные элементы разрешающего столбца
for i in range(T.shape[0]):

    if i != leaving:
        T[i] -= T[i, entering] * T[leaving]

# Обновляем базис
basis[leaving] = entering

T[np.abs(T) < EPS] = 0

```

На каждой итерации выбираются входящая и выходящая переменные, после чего определяется разрешающий элемент и выполняется пересчёт таблицы. Если подходящей выходящей переменной нет, программа определяет, что целевая функция не ограничена. Итерации заканчиваются после выполнения условия оптимальности.

Сам симплекс-алгоритм в программе реализован для задачи максимизации. Чтобы с помощью одной реализации можно было решать как задачи на максимум, так и задачи на минимум, перед второй фазой выполняется проверка направления оптимизации:

```
if sense == "min":  
    c_simplex = -c_full  
else:  
    c_simplex = c_full.copy()  
  
T = make_tableau(A, b, c_simplex, basis)  
  
T, basis = simplex(T, basis)
```

Если исходная задача задана на минимум, вместо неё внутри алгоритма максимизируется функция $-Z$. Оптимальная точка при этом не изменяется. После завершения симплекс-метода значение целевой функции рассчитывается уже по исходным коэффициентам

После запуска программы для исходной задачи был получен следующий результат:

```
Канонический вид:  
x1 + x2 + 2x4 + x5 = 8  
x2 + x3 + x4 + x7 = 6  
2x1 + x3 - x6 + x8 = 2  
  
Начальный базис: x5, x7, x8  
Искусственные переменные: x7, x8  
  
W_min = 0  
Допустимое решение существует.  
  
Оптимальное решение:  
x1 = 0  
x2 = 0  
x3 = 2  
x4 = 4  
Z_min = 8
```

Полученный программным способом результат совпадает с результатом ручного решения симплекс-методом и решением, полученным в Excel. Полный исходный код программы приведён в **приложении А**.

Вывод

В ходе выполнения работы удалось на практике разобраться с симплекс-методом и понять, как происходит переход от исходной задачи к каноническому виду и как выбираются разрешающие элементы в симплекс-таблице. Особенно полезным оказалось сравнение трёх способов решения: вручную, с помощью Excel Solver и программным способом. Это позволило проверить полученный результат разными методами.

Основные трудности возникли при реализации общего алгоритма на Python. Важно было учесть разные типы ограничений, задачи на минимум и максимум, отрицательные правые части, а также случаи, когда задача не имеет допустимого решения или является неограниченной

В результате во всех трёх способах было получено одинаковое оптимальное решение, что подтвердило правильность выполненных вычислений. Наиболее полезной частью работы для меня стала именно программная реализация, так как она помогла лучше понять логику симплекс-метода

Приложение А

```
import numpy as np

EPS = 1e-9

class Infeasible(Exception):
    pass

class Unbounded(Exception):
    pass

# ПРИВЕДЕНИЕ К КАНОНИЧЕСКОМУ ВИДУ
def canonical_form(A, b, signs):
    """
    Приведение системы ограничений к каноническому виду
    """
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    signs = list(signs)

    # Если b < 0, умножаем всё ограничение на -1
    for i in range(len(b)):
        if b[i] < 0:
            A[i] *= -1
            b[i] *= -1

            if signs[i] == "<=":
                signs[i] = ">="
            elif signs[i] == ">=":
                signs[i] = "<="

    m, n = A.shape

    names = [f"x{i + 1}" for i in range(n)]
    basis = [-1] * m
    artificial = []

    # Добавляем дополнительные переменные
    for i, sign in enumerate(signs):

        if sign in ("<=", ">="):

            column = np.zeros((m, 1))

            if sign == "<=":
                column[i] = 1
            else:
                column[i] = -1
```

```

        A = np.hstack((A, column))

        names.append(f"x{len(names) + 1}")

        # При <= новая переменная сразу базисная
        if sign == "<=":
            basis[i] = A.shape[1] - 1

    # Добавляем искусственные переменные, если в строке ещё нет базисной переменной
    for i in range(m):

        if basis[i] == -1:

            column = np.zeros((m, 1))
            column[i] = 1

            A = np.hstack((A, column))

            names.append(f"x{len(names) + 1}")

            index = A.shape[1] - 1

            basis[i] = index
            artificial.append(index)

    return A, b, basis, artificial, names

# ПОСТРОЕНИЕ СИМПЛЕКС-ТАБЛИЦЫ
def make_tableau(A, b, c, basis):
    m, n = A.shape

    T = np.zeros((m + 1, n + 1))

    T[:m, :n] = A
    T[:m, -1] = b

    # Строка целевой функции
    T[-1, :n] = -c

    # Обнуляем коэффициенты базисных переменных
    for i, j in enumerate(basis):
        T[-1] -= T[-1, j] * T[i]

    return T

# СИМПЛЕКС-МЕТОД
# Выполнение симплекс-итераций
def simplex(T, basis):

    while True:

```

```

# Ищем входящую переменную: первый отрицательный коэффициент
entering = None

for j, value in enumerate(T[-1, :-1]):
    if value < -EPS:
        entering = j
        break

# Отрицательных коэффициентов нет значит найден оптимум
if entering is None:
    return T, basis

# Выбираем выходящую переменную
ratios = []

for i in range(len(basis)):

    if T[i, entering] > EPS:

        ratio = T[i, -1] / T[i, entering]

        ratios.append((ratio, i))

# Если положительных элементов в столбце нет, целевая функция не ограничена
if not ratios:
    raise Unbounded("Целевая функция не ограничена.")

_, leaving = min(ratios)

# Разрешающий элемент
pivot = T[leaving, entering]

# Делим разрешающую строку
T[leaving] /= pivot

# Обнуляем остальные элементы разрешающего столбца
for i in range(T.shape[0]):

    if i != leaving:
        T[i] -= T[i, entering] * T[leaving]

# Обновляем базис
basis[leaving] = entering

T[np.abs(T) < EPS] = 0

# УДАЛЕНИЕ ИСКУССТВЕННЫХ ПЕРЕМЕННЫХ
def remove_artificial(T, basis, artificial, names):

    artificial = set(artificial)

```

```

i = 0

while i < len(basis):

    # Если искусственная переменная осталась в базисе
    if basis[i] in artificial:

        replacement = None

        # Ищем обычную переменную, которую можно ввести вместо неё
        for j in range(T.shape[1] - 1):

            if (
                j not in artificial
                and j not in basis
                and abs(T[i, j]) > EPS
            ):
                replacement = j
                break

        if replacement is not None:

            # Делаем новый разрешающий элемент равным 1
            T[i] /= T[i, replacement]

            # Обнуляем столбец
            for r in range(T.shape[0]):

                if r != i:
                    T[r] -= T[r, replacement] * T[i]

            basis[i] = replacement

        elif abs(T[i, -1]) < EPS:

            # Строка является избыточной
            T = np.delete(T, i, axis=0)
            basis.pop(i)

            continue

        else:

            raise Infeasible("Допустимого решения нет.")

    i += 1

# Оставляем только искусственные столбцы
keep = []

for j in range(T.shape[1] - 1):

```

```

        if j not in artificial:
            keep.append(j)

# Соответствие старых и новых индексов
old_to_new = {
    old: new
    for new, old in enumerate(keep)
}

A = T[:-1, keep]
b = T[:-1, -1]

basis = [
    old_to_new[j]
    for j in basis
]

names = [
    names[j]
    for j in keep
]

return A, b, basis, names

# ВЫВОД КАНОНИЧЕСКОГО ВИДА
def print_canonical(A, b, names):

    print("\nКанонический вид:")

    for i in range(len(b)):

        equation = ""

        for j, coef in enumerate(A[i]):

            if abs(coef) < EPS:
                continue

            value = abs(coef)

            if abs(value - 1) < EPS:
                term = names[j]
            else:
                term = f"{value:g}{names[j]}"

            if equation == "":

                if coef > 0:

```

```

        equation = term
    else:
        equation = "- " + term

    else:

        if coef > 0:
            equation += " + " + term
        else:
            equation += " - " + term

    print(f"{equation} = {b[i]:g}")

# ОСНОВНАЯ ФУНКЦИЯ РЕШЕНИЯ
def solve(c, A, signs, b, sense="min"):

# Проверка входных данных

    if sense not in ("min", "max"):
        raise ValueError("Допустимы только min и max.")

    if len(A) != len(b) or len(A) != len(signs):
        raise ValueError("Количество ограничений, знаков и ""правых частей не совпадает.")

    if any(sign not in ("<=", ">=", "=") for sign in signs):
        raise ValueError("Допустимые знаки: <=, >=, =.")

    n = len(c)

    if any(len(row) != n for row in A):
        raise ValueError("Неверное количество коэффициентов.")

    c = np.array(c, dtype=float)

# Канонический вид

    A, b, basis, artificial, names = canonical_form(A, b, signs)

    print_canonical(A, b, names)

    print(
        "\nНачальный базис:",
        ", ".join(names[j] for j in basis)
    )

    if artificial:

        print(
            "Искусственные переменные:",

```

```

        ", ".join(names[j] for j in artificial)
    )

else:

    print("Искусственные переменные не требуются.")

# ШАГ I

if artificial:

    c1 = np.zeros(A.shape[1])

    for j in artificial:
        c1[j] = -1

    T = make_tableau(A, b, c1, basis)

    T, basis = simplex(T, basis)

    # Получаем минимум вспомогательной функции
    W = -T[-1, -1]

    if abs(W) < EPS:
        W = 0

    print(f"\nW_min = {W:g}")

    # Если W > 0, исходная система несовместна
    if W > EPS:

        raise Infeasible(
            "Допустимого решения нет."
        )

    print("Допустимое решение существует.")

    # Удаляем искусственные переменные
    A, b, basis, names = remove_artificial(T, basis, artificial, names)

# ШАГ II

c_full = np.zeros(len(names))
c_full[:n] = c

if sense == "min":
    c_simplex = -c_full
else:
    c_simplex = c_full.copy()

T = make_tableau(A, b, c_simplex, basis)

```

```

T, basis = simplex(T,basis)

# ВОССТАНОВЛЕНИЕ РЕШЕНИЯ

x_all = np.zeros(len(names))

for i, j in enumerate(basis):

    x_all[j] = T[i, -1]

x = x_all[:n]
x[np.abs(x) < EPS] = 0
Z = c @ x

if abs(Z) < EPS:
    Z = 0

# Вывод результата
print("\nОптимальное решение:")

for i, value in enumerate(x):

    print(f"x{i + 1} = {value:g}")

print(f"Z_{sense} = {Z:g}")

return x, Z

# ИСХОДНЫЕ ДАННЫЕ

c = [1, 3, 2, 1]

A = [
    [1, 1, 0, 2],
    [0, 1, 1, 1],
    [2, 0, 1, 0]
]

signs = ["<=", "=", ">="]

b = [8, 6, 2]

sense = "min"

try:
    solve(
        c=c,
        A=A,

```

```
        signs=signs,  
        b=b,  
        sense=sense  
    )  
  
except Infeasible as error:  
    print("\nОшибка:", error)  
  
except Unbounded as error:  
    print("\nОшибка:", error)  
  
except ValueError as error:  
    print("\nОшибка входных данных:", error)
```